

Introduction to Python

Module 4 — Functions

Functions, scope, and list comprehensions

Why Functions?

A **function** is a reusable block of code with a name. Instead of copying the same lines everywhere, you write them once and **call** the function whenever you need it.

Functions give you

- **Reuse** — write once, use often
- **Clarity** — a name says what it does
- **Fewer bugs** — fix in one place

You've used them already

`print()`, `len()`, `type()`, `range()` are all built-in functions. Now you'll write your **own**.

Defining a Function

Use `def`, a name, **parameters** in parentheses, and `return` to send back a result.

```
def greet(name):  
    message = f"Hello, {name}!"  
    return message  
  
# call it  
greet("Denise")  # "Hello, Denise!"
```

`name` is a **parameter** — a placeholder filled in when you call the function. `return` hands back the result so you can use it.

Arguments & Return Values

Functions can take **several arguments**, and give them **default values**.

```
def power(base, exponent=2):  
    return base ** exponent  
  
power(5)      # 25 (exponent defaults to 2)  
power(5, 3)   # 125 (override the default)  
power(2, exponent=10)  # 1024 (name the argument)
```

A function **without** a return gives back None. A function can return *any* type — a number, a string, a list, a dictionary...

Scope: Variables Live Inside

Variables created *inside* a function exist only there — this is called **scope**. It keeps functions self-contained and prevents accidental clashes.

```
def add(a, b):  
    result = a + b  # 'result' lives only in here  
    return result  
  
add(2, 3)    # 5  
print(result)  # ERROR - 'result' doesn't exist out here
```

Think of a function as a room: what happens inside stays inside, unless you `return` it out.

List Comprehensions

A compact, very “Pythonic” way to build a list by transforming another. It replaces a whole for loop with one readable line.

```
words = ["cat", "dog", "bird"]

# the long way
lengths = []
for w in words:
    lengths.append(len(w))

# the comprehension way - same result
lengths = [len(w) for w in words]  # [3, 3, 4]
```

You can add a condition too: `[w for w in words if len(w) > 3]` keeps only long words. Comprehensions are everywhere in real Python.

Module 4: Takeaways

- ✓ `def name(params):` defines a reusable function
- ✓ `return` sends a result back; no return means `None`
- ✓ Arguments can have **defaults** and be passed **by name**
- ✓ Variables inside a function have local **scope**
- ✓ **List comprehensions** `[f(x) for x in xs]` build lists compactly

Next: Module 5 — working with **strings and files** in depth.